



Faculty of Engineering & Technology

Electrical & Computer Engineering Department

Advanced Digital Design - ENCS4310

Smart Traffic Light Controller with Pedestrian Request (FSM)

Project -Report

Prepared by:

Sondos Hammad 1230521

Instructor:

Dr.Abd-Allatef Abu Issa

Section: 4

Date: 9/Jan/2026

Abstract:

This project implements a finite state machine (FSM) traffic controller for a main road and a side road with a pedestrian push button. The design follows fixed timing constraints and supports queued requests from the vehicle sensor (VS) and pedestrian button (PB). A reference model and a self-checking testbench are used to verify that the RTL behavior is correct.

Table of contents

Abstract:	I
Table of contents	II
Table of figures	III
Table of tables	III
System flow	1
State diagram	1
States and colors Encoding	2
Colors encoding	2
States encoding	2
Transition table	3
State Transition Table	3
Output logic table	3
Code implementation	4
Traffic_ctrl code	4
Traffic_ref code	4
Test-bench of both codes	5
Reference code	8
Conclusion	10

Table of figures

Figure 1 : State Diagram	1
Figure 2 : Test-bench default case	5
Figure 3 : Test-bench PB request	5
Figure 4 : Test-bench PB request cont	5
Figure 5 : Test-bench VS execution	6
Figure 6 : Test-bench PB execution	6
Figure 7 : Test-bench VS execution	6
Figure 8 : Test-bench two requests at the same time	7
Figure 9 : Test-bench PB execution	7
Figure 10 : Test-bench default case	7
Figure 11 : Test-bench with reset	8
Figure 12 : Test-bench console output	8

Table of tables

Table 1 : State transition table	3
Table 2 : Output logic table	3

System flow

There are two roads: a main road and a side road. The default state is Main Green for 60 seconds while the side road remains Red. If a request occurs during this period (either from the vehicle sensor VS or the pedestrian push button PB), the controller waits until the 60 seconds finish, then transitions to Main Yellow for 4 seconds (assumed), followed by an All-Red clearance state for 2 seconds (assumed). After the all-red interval, the controller serves the pending request (vehicle request or pedestrian walk request).

State diagram

After reading the project specification and attempting the implementation, I first analyzed the text and rewrote it as a clear set of requirements. Then, based on that analysis, I designed the FSM state diagram shown below.

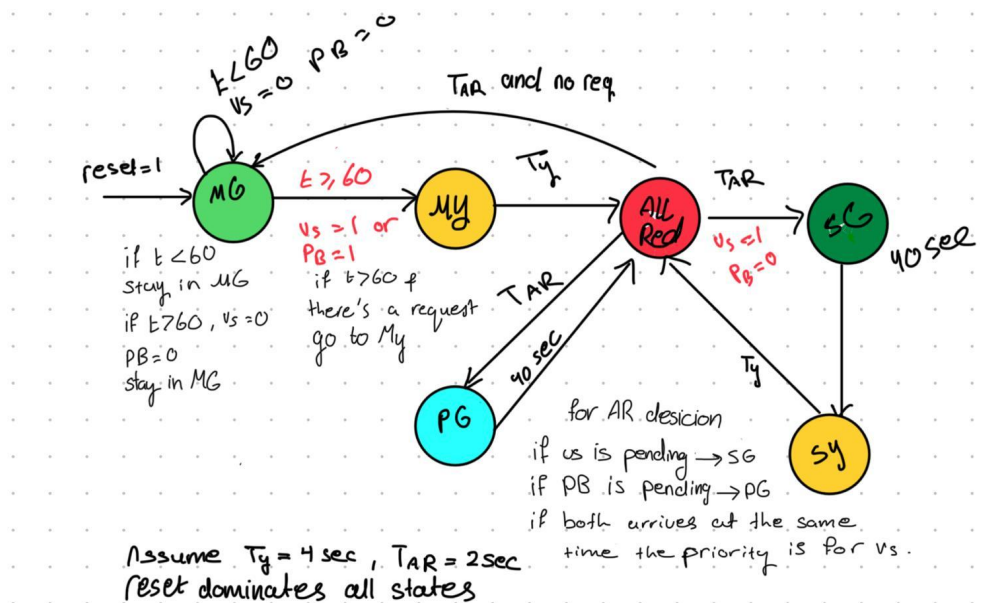


Figure 1: State Diagram

The diagram directly matches the RTL code in `traffic_ctrl`. Each circle represents one encoded state in the design (ST_MG, ST_MY, ST_AR, ST_SG, ST_SY, ST_PG). The controller starts in Main Green (MG) after reset, where the timer counter `t` runs for 60 seconds. If a request is detected, the FSM transitions to Main Yellow (MY) for 4 seconds, then to All Red (AR) for 2 seconds as a clearance interval. In the AR state, the code checks the pending request flags (`vs_pending`, `pb_pending`) and selects the next service state: Side Green (SG) for vehicle requests or Pedestrian Walk (PG) for pedestrian requests. If both requests arrive, the priority rule shown in the diagram is implemented using `first_is_vs`, so VS is served first when simultaneous. After servicing, the FSM returns through Side Yellow (SY) and All Red (AR) before going back to Main Green, ensuring safe transitions exactly as represented in the diagram.

States and colors Encoding

Colors encoding

The traffic light outputs are encoded using a **one-hot color representation** in the form {RED, YEL, GRN}. This means that at any time only one bit is asserted, which directly matches the real-world behavior of a traffic light (a lamp is either red, yellow, or green). Using one-hot encoding makes the output logic clear and safe, because it prevents invalid combinations such as turning on two colors at the same time, since each color corresponds to a unique 3-bit pattern:

Red = 3'b100 {RED, YEL, GRN}

Yellow = 3'b010 {RED, YEL, GRN}

Green = 3'b001 {RED, YEL, GRN}

States encoding

The FSM in traffic_ctrl uses **binary state encoding** because the controller has only six states, and binary encoding can represent them using just 3 bits (3 flip-flops). This makes the implementation more compact and efficient, each state is assigned a unique 3-bit value, and the transitions are handled through the state and next_state signals. This choice keeps the hardware simple.

ST_MG (Main Green) = 3'd0 → 3'b000

ST_MY (Main Yellow) = 3'd1 → 3'b001

ST_AR (All Red) = 3'd2 → 3'b010

ST_SG (Side Green) = 3'd3 → 3'b011

ST_SY (Side Yellow) = 3'd4 → 3'b100

ST_PG (Pedestrian Walk) = 3'd5 → 3'b101

Transition table

State Transition Table

Current State	Condition	Next State
ST_MG	Timer not done	ST_MG
ST_MG	Timer done & no request (vs_pending=0 & pb_pending=0)	ST_MG
ST_MG	Timer done & any request (vs_pending=1 or pb_pending=1)	ST_MY
ST_MY	Timer not done	ST_MY
ST_MY	Timer done	ST_AR
ST_AR	Timer not done	ST_AR
ST_AR	Timer done AND (vs_pending=1 AND pb_pending=1)	first_is_vs ? ST_SG : ST_PG
ST_AR	Timer done AND (vs_pending=1 AND pb_pending=0)	ST_SG
ST_AR	Timer done AND (vs_pending=0 AND pb_pending=1)	ST_PG
ST_AR	Timer done AND no pending requests	ST_MG
ST_SG	Timer not done	ST_SG
ST_SG	Timer done	ST_SY
ST_SY	Timer not done	ST_SY
ST_SY	Timer done	ST_AR
ST_PG	Timer not done	ST_PG
ST_PG	Timer done	ST_AR

Table 1: State transition table

Output logic table

State	Main Light	Side Light	Walk
ST_MG	Green	Red	0
ST_MY	Yellow	Red	0
ST_AR	Red	Red	0
ST_SG	Red	Green	0
ST_SY	Red	Yellow	0
ST_PG	Red	Red	1

Table 2: Output logic table

Code implementation

Traffic_ctrl code

This Verilog module implements a timed traffic light FSM controller for a main road, a side road, and a pedestrian crossing. The design uses binary state encoding with six states (ST_MG, ST_MY, ST_AR, ST_SG, ST_SY, ST_PG) stored in the state register, while next_state defines the transition behavior. A timer counter t is used to measure the time spent in each phase, and state transitions occur when t reaches the required limits (Main Green transitions after 60 seconds, Yellow after 4 seconds, All-Red after 2 seconds, and both Side Green and Pedestrian Walk after 40 seconds). Requests from the vehicle sensor (VS) and the pedestrian push button (PB) are stored using pending flags (vs_pending, pb_pending) to ensure that short pulses are not missed. If both requests are pending, the controller applies a **first come first served policy using first_is_vs**, and when both arrive at the same time, VS is served first. The decision of which request to serve is made in the All-Red state, where the FSM selects either Side Green to serve vehicles or Pedestrian Walk to serve pedestrians. Finally, the outputs are Moore-style: light_main, light_side, and walk depend only on the current state, producing the correct traffic signals for each phase.

Traffic_ref code

This module (traffic_ref) is written as a behavioral reference model, so it does not rely on the FSM diagram or explicit state encoding using state/next_state. Instead, it determines where the controller currently is by checking the current output combination (light_main, light_side, and walk), and then decides where it should go next based on the specification rules. The timing of each phase is handled using the counter t, which tracks how long the controller remains in the same phase, and transitions occur only when the required time for that phase is completed (Main Green 60 s, Yellow 4 s, All Red 2 s, Side Green 40 s, and Pedestrian Walk 40 s). In addition, requests from the vehicle sensor (VS) and the pedestrian push button (PB) are stored using vs_pending and pb_pending to ensure that no request is lost while the controller is busy. When both requests are pending, the variable first_is_vs is used to apply the priority arrival rule (simultaneous requests serve VS first, otherwise the first arriving request is served first). Overall, this reference model provides a clear expected behavior sequence which is then used as the baseline to verify the RTL implementation in traffic_ctrl using the self-checking testbench.

Test-bench of both codes

At first, I implemented a simple testbench for the reference module to confirm that it behaves correctly. This testbench is attached at the end of the report for completeness, although it is not essential to the main verification flow.

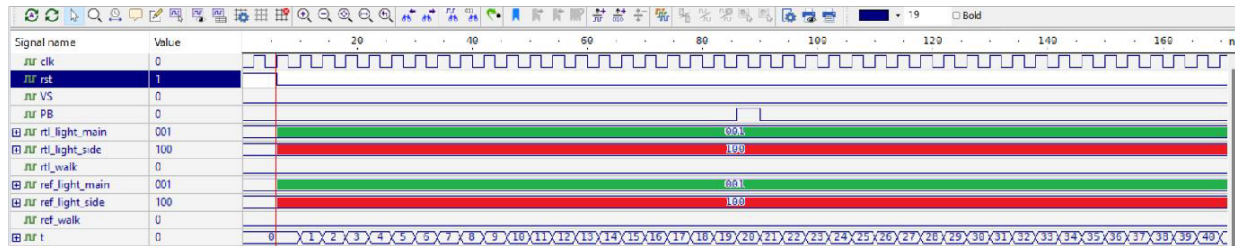


Figure 2: Test-bench default case

At the beginning, $rst = 1$. After the reset becomes 0, the controller enters the main green / side red state and the timer starts counting 0–59. During this main-green interval, a pedestrian request (pb) occurs and is stored as a pending request.

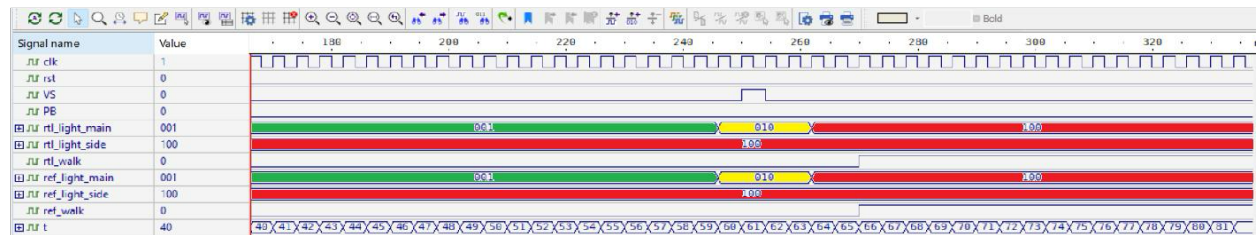


Figure 3: Test-bench PB request

When the main green time completes, the controller transitions to main yellow for 4 seconds ($t = 60-63$), and during the yellow interval a vehicle request (VS) occurs and is also stored as a pending request. then moves to the all-red state for 2 seconds ($t = 64-65$). After the all-red clearance, the controller serves the pending (PB) request by enabling the walk signal.

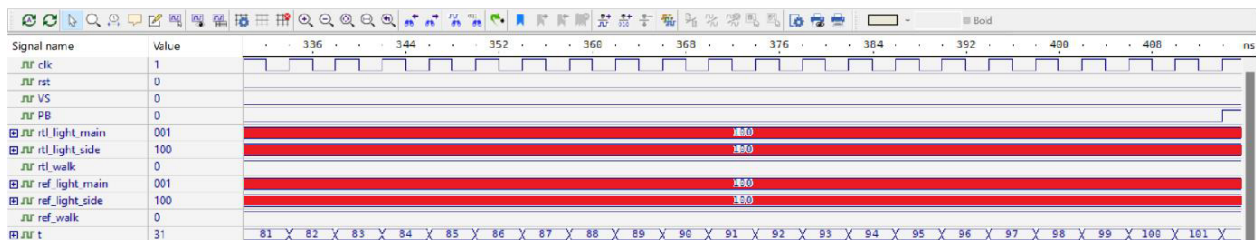


Figure 4 : Test-bench PB request cont

During this phase, both traffic lights remain red, and the walk output stays on for 40 seconds($t= 66-105$).

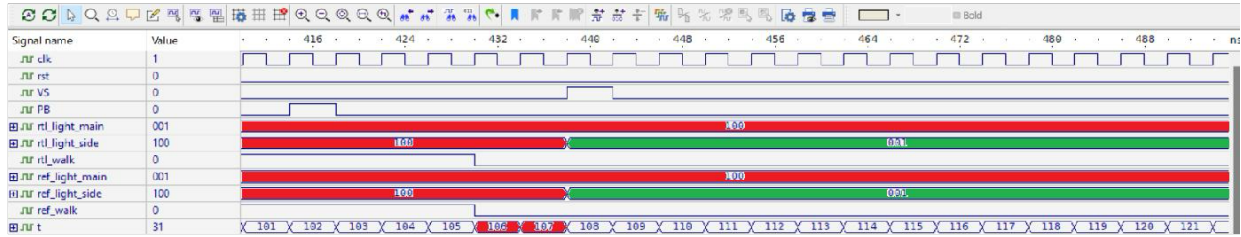


Figure 5: Test-bench VS execution

During the 40 s pedestrian walk phase (66–105), another pb request and after a small time another vs request occurs, and a vs request is also pending. After the walk interval ends, the controller enters the all-red state for 2 seconds. Then it serves the pending vs request by switching to side green and main red for 40 seconds (103–142).

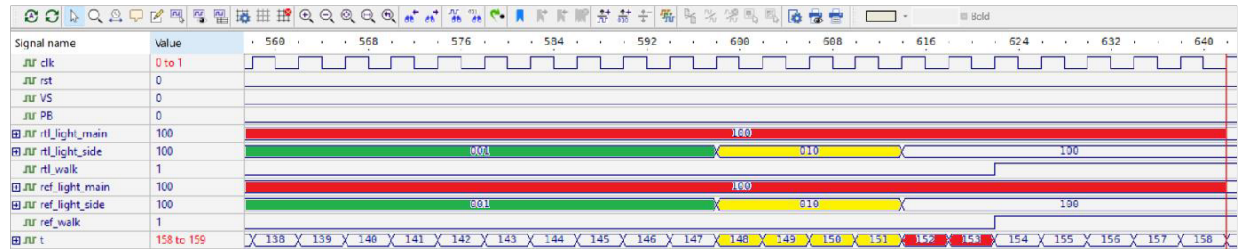


Figure 6 : Test-bench PB execution

After executing the vs request side green for 40 sec & main red from (108–147), it will go to side yellow for 4 sec, then all red for 2 sec. After that, the first pending request was pb, so it will start executing it by making main and side red for 40 sec (154–193) and walk = 1.

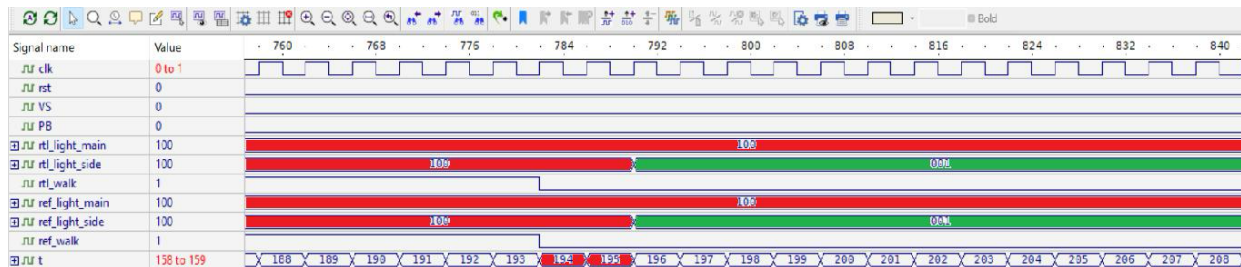


Figure 7 : Test-bench VS execution

After executing the PB request, both main and side are red and walk = 1 for 40 sec from (154–193). Then it will go to all red for 2 sec, and after that there is a pending VS request, so it will execute side green (main red) for 40 sec from (196–235).

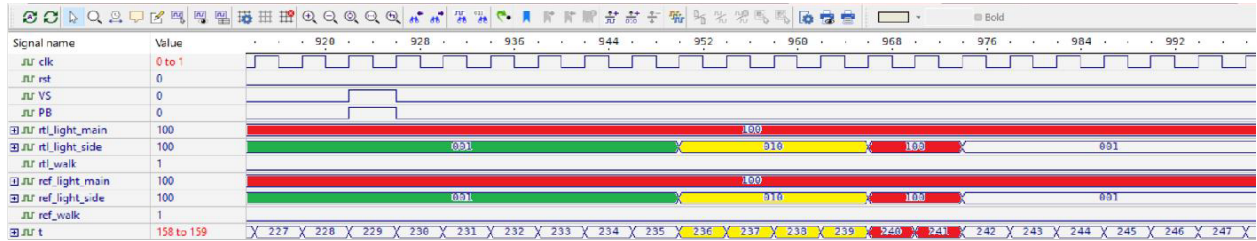


Figure 8 : Test-bench two requests at the same time

During execution of the VS request, the side is green and the main is red from (196–235). During this time, two requests **arrive at the same time** (VS and PB), so both become pending. After finishing the VS phase, it goes to yellow and then all red. Since two requests came at the same time, **I set the priority to execute VS first**, so it starts executing VS again with side green and main red for 40 sec from (242–281).

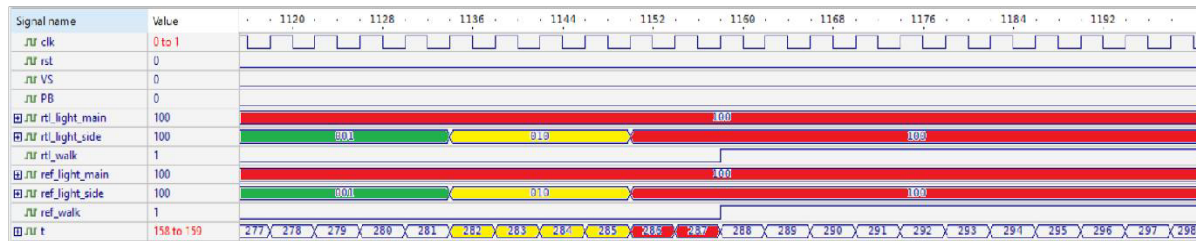


Figure 9 : Test-bench PB execution

After executing the VS request (side green and main red) for 40 sec, it will go to yellow for 4 sec, then to all red for 2 sec. After that, it will serve the pending PB request by making both main and side red and setting walk = 1 for 40 sec from (288–327). And after that since no requests it will stay on the default case main green and side red .

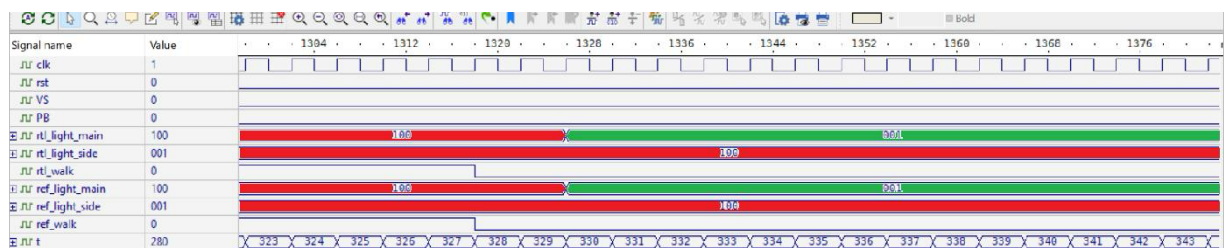


Figure 10 :Test-bench default case

After that, I edited the testbench to add a reset pulse. So after executing the PB request and entering the 2 second all-red state, the reset is asserted. This forces the controller back to its default state, where the main road is green and the side road is red (walk = 0).

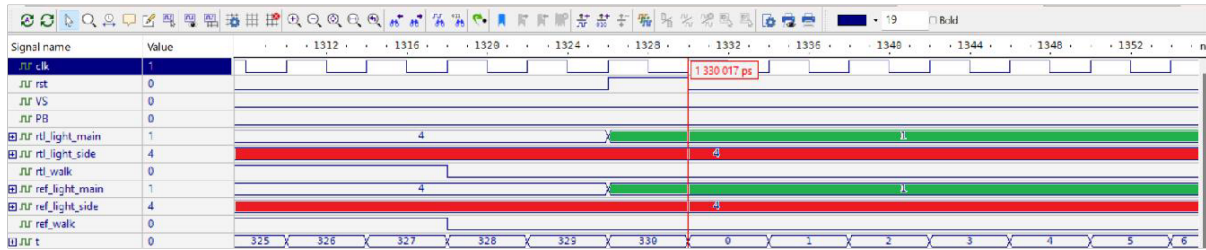


Figure 11 : Test-bench with reset

The simulation completed successfully: all test cases passed, and the RTL outputs match the reference model (**PASS: RTL matches REF**). No mismatches were detected during the run.

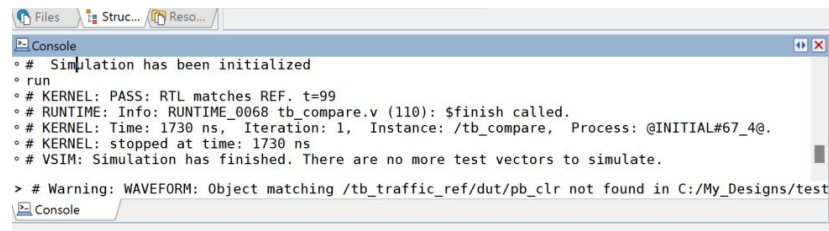
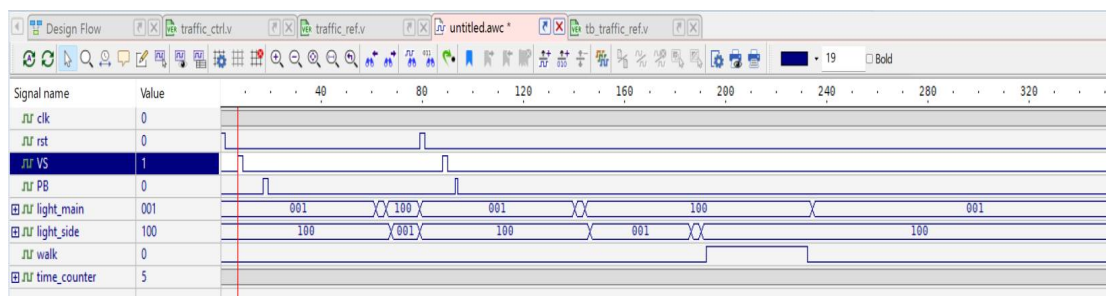


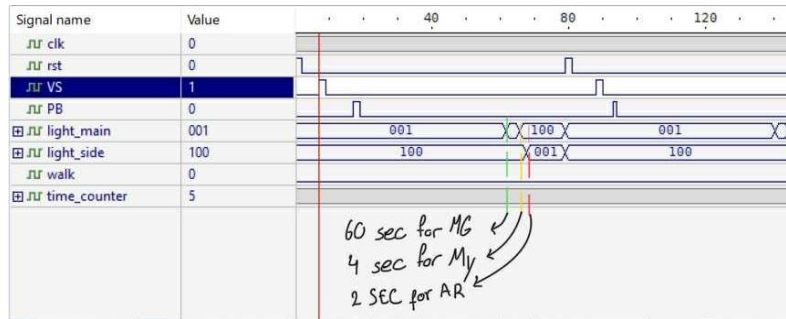
Figure 12 : Test-bench console output

Reference code

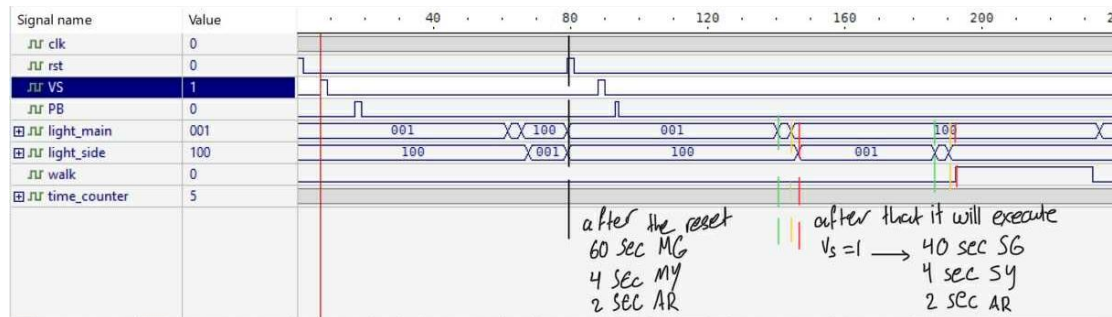
The testbench for this code was implemented before the test bench for both to check that the reference code work 100% true



The figure illustrates the behavior of the traffic_ref controller. At the beginning, the main road light is green (001) while the side road light is red. A vehicle is detected on the side road, so VS becomes 1, but since the main road green period has not yet completed its 60-second duration, the controller keeps the main light green until the timer finishes. Once the 60 seconds expire, the main light transitions to yellow for 4 seconds, then both directions switch to all-red for 2 seconds before the next phase begins.



Next, since $VS = 1$, the controller should grant the side road makes the main light red (100) and the side light green (001) for 40 seconds. However, a reset is asserted at that moment, which reinitializes the system and forces it back to the default state: main green (001) and side red (100), restarting the 60 second main green interval.



After the reset, VS becomes 1 again. Once the controller completes the 60 second main green interval, the main light transitions to yellow for 4 seconds, then to all red for 2 seconds. After that, since $VS = 1$, the side road is served: the side light turns green for 40 seconds while the main light stays red. When the 40 second side green period ends, the side light changes to yellow for 4 seconds, followed by all red for 2 seconds.

During execution, PB becomes 1, so the pedestrian request is served. In this case, both the main and side lights remain red, while the walk signal is asserted for 40 seconds (this is the width of the walk pulse).

Conclusion

In this project, a complete traffic light controller was designed using an FSM with fixed timing requirements and support for both vehicle and pedestrian requests. The system follows the required sequence of phases (Main Green, Yellow, All Red, Side Green or Walk) while ensuring that requests are not missed by using pending flags. A behavioral reference model was also developed to represent the expected behavior without relying on explicit state encoding, and it was used as a baseline for verification. Finally, a self-checking testbench compared the RTL implementation against the reference model and confirmed correct transitions, timing behavior, reset operation, and request handling priority through waveform results.